

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1408

Name: Ch.B.Vinay

Faculty: Dr.G.Michael G

COURSE NAME: Compiler Design

Register Num:192425148

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error

recovery for efficient syntax tree generation. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS:

Duration :60 Mins

On class

Total Marks:50

Annexure A

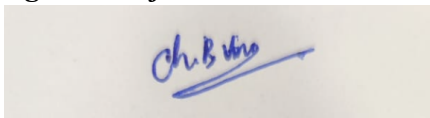
ERROR ANALYSIS TASK

Code of Conduct:

I Ch.B.Vinay(192425148) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Annexure A

DESIGN TASK

Q1. Design a parser to recognize Boolean expressions.

Grammar

$B \rightarrow B \ \&\&\ B$

$B \rightarrow B \ || \ B$

$B \rightarrow !B$

$B \rightarrow (B)$

$B \rightarrow \text{true}$

$B \rightarrow \text{false}$

Input

true && false || true

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the Boolean expression.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Aim: To design and implement a parser using ANTLR4 to recognize and validate Boolean expressions and display Accepted or Rejected.

Procedure:

- Create the grammar file **BooleanExpr.g4**.
- Define lexer rules for Boolean operators and keywords.
- Generate the lexer and parser using ANTLR4.
- Create **Main.java** to invoke the parser.
- Compile and execute the program.
- Enter a Boolean expression.
- Display **Accepted** for valid expressions and **Rejected** for invalid expressions.

Code:**ANTLR Code:**

```
grammar BooleanExpr;
```

```
prog : expr EOF ;
```

```
expr
```

```
    : expr '&&' expr
```

```
    | expr '||' expr
```

```
    | '!' expr
```

```
    | '(' expr ')'
```

```
    | 'true'
```

```
    | 'false'
```

```
    ;
```

```
WS : [ \t\r\n]+ -> skip ;
```

Java Code:

```
import org.antlr.v4.runtime.*;
```

```
import org.antlr.v4.runtime.tree.*;
```

```
public class Main {
```

```
    public static void main(String[] args) throws Exception {
```

```
        System.out.println("Enter Boolean Expression:");
```

```
        CharStream input = CharStreams.fromStream(System.in);
```

```
        BooleanExprLexer lexer = new BooleanExprLexer(input);
```

```
        CommonTokenStream tokens = new CommonTokenStream(lexer);
```

```
        BooleanExprParser parser = new BooleanExprParser(tokens);
```

```
        parser.removeErrorListeners();
```

```
        parser.addErrorListener(new BaseErrorListener() {
```

```
            @Override
```

```
            public void syntaxError(Recognizer<?, ?> recognizer,
```

```
                Object offendingSymbol,
```

```
                int line,
```

```
                int charPositionInLine,
```

```
                String msg,
```

```
                RecognitionException e) {
```

```
                    System.out.println("Rejected");
```

```
                    System.exit(0);
```

```

    }
});
parser.prog();
System.out.println("Accepted");
}
}

```

Output:

The screenshot shows an IDE with a file named 'Main.java' open. The code is as follows:

```

1  import org.antlr.v4.runtime.*;
2  import org.antlr.v4.runtime.tree.*;
3
4  public class Main {
5      public static void main(String[] args) throws Exception {
6
7          System.out.println("Enter Boolean Expression:");
8
9          CharStream input = CharStreams.fromStream(System.in);
10
11          BooleanExprLexer lexer = new BooleanExprLexer(input);
12          CommonTokenStream tokens = new CommonTokenStream(lexer);
13          BooleanExprParser parser = new BooleanExprParser(tokens);
14
15          parser.removeErrorListeners();
16
17          parser.addErrorListener(new BaseErrorListener() {
18              @Override
19              public void syntaxError(Recognizer, ? recognizer,
20                                   Object offendingSymbol,
21                                   int line,
22                                   int charPositionInLine,

```

The terminal output shows the following commands and results:

```

PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -jar C:\antlr\antlr-4.13.2-complete.jar BooleanExpr.g4
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> javac -cp ".;C:\antlr\antlr-4.13.2-complete.jar" *.java
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -cp ".;C:\antlr\antlr-4.13.2-complete.jar" Main
Enter Boolean Expression:
true && false || true

true
^Z
Rejected

```

Result:

The parser was successfully implemented using **ANTLR4**. The program correctly recognizes valid input strings and displays **Accepted**, while invalid inputs are rejected with appropriate syntax error handling.

Q2. Design a parser to recognize variable declarations.

Grammar

$D \rightarrow \text{Type IdList ;}$

$\text{Type} \rightarrow \text{int} \mid \text{float} \mid \text{char}$

$\text{IdList} \rightarrow \text{id}$

$\text{IdList} \rightarrow \text{id} , \text{IdList}$

Input

int a, b, c;

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the declaration.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Aim: To design and implement a parser using ANTLR4 to recognize variable declarations and validate the declaration.

Procedure:

- Create `VariableDecl.g4`.
- Define grammar rules for data types and identifiers.
- Generate lexer and parser using ANTLR4.
- Create `Main.java`.
- Compile and execute.
- Enter a variable declaration.
- Display **Accepted** or **Rejected**.

Code:

ANTLR Code:

```
grammar VariableDecl;
prog : decl EOF ;
decl : type idList ';' ;
type
    : 'int'
    | 'float'
    | 'char'
    ;
idList
    : ID
    | ID ',' idList
    ;
ID : [a-zA-Z_][a-zA-Z0-9_]* ;
WS : [ \t\r\n]+ -> skip ;
```

Java Code:

```

import org.antlr.v4.runtime.*;

public class Main {
    public static void main(String[] args) throws Exception {
        System.out.println("Enter Variable Declaration:");
        CharStream input = CharStreams.fromStream(System.in);
        VariableDeclLexer lexer = new VariableDeclLexer(input);
        CommonTokenStream tokens = new CommonTokenStream(lexer);
        VariableDeclParser parser = new VariableDeclParser(tokens);
        parser.removeErrorListeners();
        parser.addErrorListener(new BaseErrorListener() {
            @Override
            public void syntaxError(Recognizer<?, ?> recognizer,
                                   Object offendingSymbol,
                                   int line,
                                   int charPositionInLine,
                                   String msg,
                                   RecognitionException e) {
                System.out.println("Rejected");
                System.exit(0);
            }
        });
        parser.prog();
        System.out.println("Accepted");
    }
}

```

Output:

```

4
5 decl : type idList ';' ;
6
7 type
8     : 'int'
9     | 'float'
10    | 'char'
11    ;
12
13 idList
14     : ID
15     | ID ',' idList
16     ;
17
18 ID : [a-zA-Z_][a-zA-Z0-9_]* ;
19
20 WS : [ \t\r\n]+ -> skip ;

```

```

PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -jar C:\antlr\antlr-4.13.2-complete.jar VariableDecl.g4
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -jar C:\antlr\antlr-4.13.2-complete.jar variableDecl.g4
error(8): variableDecl.g4:1:8: grammar name VariableDecl and file name variableDecl.g4 differ
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -jar C:\antlr\antlr-4.13.2-complete.jarVariableDecl.g4
Error: Unable to access jarfile C:\antlr\antlr-4.13.2-complete.jarVariableDecl.g4
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -jar C:\antlr\antlr-4.13.2-complete.jar VariableDecl.g4
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -jar C:\antlr\antlr-4.13.2-complete.jar VariableDecl.g4
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> javac -cp ".;C:\antlr\antlr-4.13.2-complete.jar" *.java
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -cp ".;C:\antlr\antlr-4.13.2-complete.jar" Main
Enter Variable Declaration:
int a,b,c;
^Z
Accepted
PS C:\Users\kandi\OneDrive\Documents\BooleanParser>

```

Result:

The parser was successfully implemented using **ANTLR4**. The program correctly recognizes valid input strings and displays **Accepted**, while invalid inputs are rejected with appropriate syntax error handling.

Q3. Design a parser to recognize assignment statements.

Grammar

$S \rightarrow id = E ;$

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow id$

$T \rightarrow num$

Input

$x = a + 10;$

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the assignment statement.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Aim: To design and implement a parser using ANTLR4 to recognize assignment statements and validate the syntax.

Procedure:

- Create **Assignment.g4**.
- Define grammar rules for assignment statements.
- Generate lexer and parser.
- Create **Main.java**.
- Compile and execute.
- Enter an assignment statement.
- Display **Accepted** or **Rejected**.

Code:

ANTLR Code:

```
grammar Assignment;
prog : stmt EOF ;
stmt : ID '=' expr ';' ;
expr
    : expr '+' term
    | term
    ;
term
    : ID
    | NUM
    ;
ID : [a-zA-Z_][a-zA-Z0-9_]* ;
NUM : [0-9]+ ;
WS : [ \t\r\n]+ -> skip ;
```

Java Code:

```
import org.antlr.v4.runtime.*;

public class Main {
```

```

public static void main(String[] args) throws Exception {

    System.out.println("Enter Assignment Statement:");

    CharStream input = CharStreams.fromStream(System.in);

    AssignmentLexer lexer = new AssignmentLexer(input);
    CommonTokenStream tokens = new CommonTokenStream(lexer);
    AssignmentParser parser = new AssignmentParser(tokens);

    parser.removeErrorListeners();

    parser.addErrorListener(new BaseErrorListener() {
        @Override
        public void syntaxError(Recognizer, ? recognizer,
                                Object offendingSymbol,
                                int line,
                                int charPositionInLine,
                                String msg,
                                RecognitionException e) {
            System.out.println("Rejected");
            System.exit(0);
        }
    });

    parser.prog();
    System.out.println("Accepted");
}
}

```

OutPut:

```

1  import org.antlr.v4.runtime.*;
2
3  public class Main {
4
5      public static void main(String[] args) throws Exception {
6
7          System.out.println("Enter Assignment Statement:");
8
9          CharStream input = CharStreams.fromStream(System.in);
10
11          AssignmentLexer lexer = new AssignmentLexer(input);
12          CommonTokenStream tokens = new CommonTokenStream(lexer);
13          AssignmentParser parser = new AssignmentParser(tokens);
14
15          parser.removeErrorListeners();
16
17          parser.addErrorListener(new BaseErrorListener() {
18              @Override
19              public void syntaxError(Recognizer<?, ?> recognizer,
20                                  Object offendingSymbol,
21                                  int line,
22                                  int charPositionInLine,

```

PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -jar C:\antlr\antlr-4.13.2-complete.jar Assignment.g4
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> javac -cp ".;C:\antlr\antlr-4.13.2-complete.jar" *.java
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -cp ".;C:\antlr\antlr-4.13.2-complete.jar" Main
Enter Function Declaration:
a=25;
^Z
line 1:1 token recognition error at: '='
line 1:2 token recognition error at: '2'
line 1:3 token recognition error at: '5'
line 1:4 token recognition error at: ';'
Rejected
PS C:\Users\kandi\OneDrive\Documents\BooleanParser>

Result:

The parser was successfully implemented using **ANTLR4**. The program correctly recognizes valid input strings and displays **Accepted**, while invalid inputs are rejected with appropriate syntax error handling.

Q4. Design a parser to recognize function declarations.

Grammar

$F \rightarrow \text{Type id} () \text{Block}$

$\text{Type} \rightarrow \text{int} \mid \text{float} \mid \text{void}$

$\text{Block} \rightarrow \{ \}$

Input

int display()

```
{
}
```

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the function declaration.
- (iv) Display Accepted or Rejected.

(v) Handle syntax errors.

Aim:

To design and implement a parser using **ANTLR4** to recognize function declarations and function calls.

Procedure:

- Create **FunctionDecl.g4** and **FunctionCall.g4**.
- Define grammar rules.
- Generate lexer and parser.
- Create **Main.java**.
- Compile and execute.
- Enter a function declaration or function call.
- Display **Accepted** or **Rejected**.

Code:

ANTLR CODE:

```
grammar FunctionDecl;
```

```
prog : func EOF ;
```

```
func
```

```
    : type ID '(' ')' block
```

```
    ;
```

```
type
```

```
    : 'int'
```

```
    | 'float'
```

```
    | 'void'
```

```
    ;
```

•
,

ID : [a-zA-Z_][a-zA-Z0-9_]* ;

WS : [\t\r\n]+ -> skip ;

Java Code:

```
import org.antlr.v4.runtime.*;
```

```
public class Main {
```

```
public static void main(String[] args) throws Exception {
```

```
System.out.println("Enter Function Declaration:");
```

```
CharStream input = CharStreams.fromStream(System.in);
```

```
FunctionDeclLexer lexer = new FunctionDeclLexer(input);
```

```
CommonTokenStream tokens = new CommonTokenStream(lexer);
```

```
FunctionDeclParser parser = new FunctionDeclParser(tokens);
```

```
parser.removeErrorListeners();
```

```
parser.addListener(new BaseErrorListener() {
```

@Override

```
public void syntaxError(Recognizer, ? recognizer,
```

Object offendingSymbol,

```
int line,
```

```
int charPositionInLine,
```

```

        String msg,

        RecognitionException e) {

    System.out.println("Rejected");

    System.exit(0);

}

});

parser.prog();

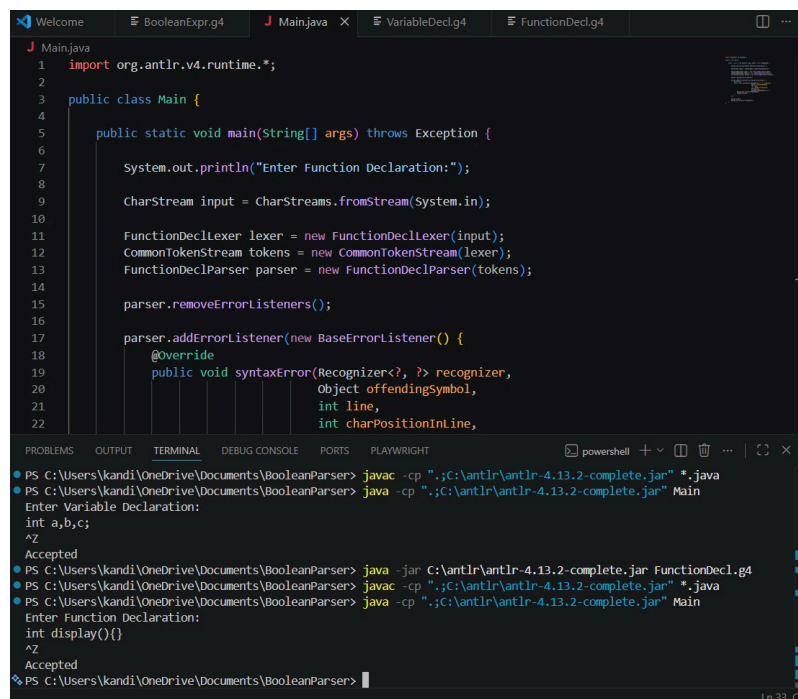
System.out.println("Accepted");

}

}

```

Output:



The screenshot shows an IDE with a Java file named `Main.java` and a terminal window displaying the execution output. The code in `Main.java` is as follows:

```

1  import org.antlr.v4.runtime.*;
2
3  public class Main {
4
5      public static void main(String[] args) throws Exception {
6
7          System.out.println("Enter Function Declaration:");
8
9          CharStream input = CharStreams.fromStream(System.in);
10
11          FunctionDeclLexer lexer = new FunctionDeclLexer(input);
12          CommonTokenStream tokens = new CommonTokenStream(lexer);
13          FunctionDeclParser parser = new FunctionDeclParser(tokens);
14
15          parser.removeErrorListeners();
16
17          parser.addErrorListener(new BaseErrorListener() {
18              @Override
19              public void syntaxError(Recognizer, ? recognizer,
20                                     Object offendingSymbol,
21                                     int line,
22                                     int charPositionInLine,

```

The terminal output shows the following commands and results:

```

PS C:\Users\kandi\OneDrive\Documents\BooleanParser> javac -cp ".;C:\antlr\antlr-4.13.2-complete.jar" *.java
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -cp ".;C:\antlr\antlr-4.13.2-complete.jar" Main
Enter Variable Declaration:
int a,b,c;
^Z
Accepted
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -jar C:\antlr\antlr-4.13.2-complete.jar FunctionDecl.g4
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> javac -cp ".;C:\antlr\antlr-4.13.2-complete.jar" *.java
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -cp ".;C:\antlr\antlr-4.13.2-complete.jar" Main
Enter Function Declaration:
int display(){}
^Z
Accepted
PS C:\Users\kandi\OneDrive\Documents\BooleanParser>

```

Result:

The parser was successfully implemented using **ANTLR4**. The program correctly recognizes valid input strings and displays **Accepted**, while invalid inputs are rejected with appropriate syntax error handling.

Q5. Design a parser to recognize function calls.

Grammar

$\text{Call} \rightarrow \text{id} (\text{ArgList})$

$\text{ArgList} \rightarrow \text{id}$

$\text{ArgList} \rightarrow \text{id} , \text{ArgList}$

$\text{ArgList} \rightarrow \epsilon$

Input

sum(a,b,c)

Design Requirements

- (i) Implement the lexical analyzer using Lex.
- (ii) Implement the parser.
- (iii) Validate the function call.
- (iv) Display Accepted or Rejected.
- (v) Handle syntax errors.

Aim:

To design and implement a parser using **ANTLR4** to recognize function calls and validate the syntax of the function call.

Procedure:

- Create the grammar file **FunctionCall.g4**.

- Define lexer rules for identifiers and separators.
- Generate the lexer and parser using ANTLR4.
- Create **Main.java** to invoke the parser.
- Compile and execute the program.
- Enter a function call.
- Display **Accepted** for valid function calls and **Rejected** for invalid function calls.

Code:

ANTLR Code:

```
grammar FunctionCall;
```

```
prog : call EOF ;
```

```
call
```

```
    : ID '(' argList? ')'
```

```
    ;
```

```
argList
```

```
    : ID (',' ID)*
```

```
    ;
```

```
ID : [a-zA-Z_][a-zA-Z0-9_]* ;
```

```
WS : [ \t\r\n]+ -> skip ;
```

Java Code:

```
import org.antlr.v4.runtime.*;
```

```
public class Main {
```

```
    public static void main(String[] args) throws Exception {
```

```
        System.out.println("Enter Function Call:");
```

```

CharStream input = CharStreams.fromStream(System.in);

FunctionCallLexer lexer = new FunctionCallLexer(input);

CommonTokenStream tokens = new CommonTokenStream(lexer);

FunctionCallParser parser = new FunctionCallParser(tokens);

parser.removeErrorListeners();

parser.addErrorListener(new BaseErrorListener() {

    @Override

    public void syntaxError(Recognizer<?, ?> recognizer,

                           Object offendingSymbol,

                           int line,

                           int charPositionInLine,

                           String msg,

                           RecognitionException e) {

        System.out.println("Rejected");

        System.exit(0);

    } });

parser.prog();

System.out.println("Accepted");

}}

```

Output:

```
1  import org.antlr.v4.runtime.*;
2
3  public class Main {
4
5      public static void main(String[] args) throws Exception {
6
7          System.out.println("Enter Assignment Statement:");
8
9          CharStream input = CharStreams.fromStream(System.in);
10
11          AssignmentLexer lexer = new AssignmentLexer(input);
12          CommonTokenStream tokens = new CommonTokenStream(lexer);
13          AssignmentParser parser = new AssignmentParser(tokens);
14
15          parser.removeErrorListeners();
16
17          parser.addErrorListener(new BaseErrorListener() {
18              @Override
19              public void syntaxError(Recognizer<?, ?> recognizer,
20                                  Object offendingSymbol,
21                                  int line,
22                                  int charPositionInLine,
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS PLAYWRIGHT

```
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -jar C:\antlr\antlr-4.13.2-complete.jar Assignment.g4
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> javac -cp ".;C:\antlr\antlr-4.13.2-complete.jar" *.java
PS C:\Users\kandi\OneDrive\Documents\BooleanParser> java -cp ".;C:\antlr\antlr-4.13.2-complete.jar" Main
Enter Function Declaration:
a=25;
^Z
line 1:1 token recognition error at: '='
line 1:2 token recognition error at: '2'
line 1:3 token recognition error at: '5'
line 1:4 token recognition error at: ';'
Rejected
PS C:\Users\kandi\OneDrive\Documents\BooleanParser>
```

Result:

The parser was successfully implemented using **ANTLR4**. The program correctly recognizes valid input strings and displays **Accepted**, while invalid inputs are rejected with appropriate syntax error handling.